

OpenADMET-ExpansionRx Blind Challenge — 复现 + 改进实验完整报告

项目：[OpenADMET-LightGBM-Reproduction/](#)

作者：Yuze Li

日期：2026-05-03 (v1/v2) / 2026-05-05 (v3) / 2026-05-07 (v3.1)

最佳成绩：MA-RAE = 0.666 (v3 with 10 seeds × 80 epochs, threshold=1.10)

历史最佳记录：

- v1 + selective augmentation = 0.750 (CPU only, 5 分钟)
- v3 (3-cluster, 5 seeds, 50ep) = 0.677 (T4, ~90 分钟)
- v3 (3-cluster, 10 seeds, 80ep) = 0.666 (T4, ~3 小时) ← 历史最佳
- v3.1 (HybridADMET + CheMeleon, 5 seeds, 50ep) = 0.679 (T4, ~6 小时) ← 没改善, 详见 §12.7-12.10

0. TL;DR

- 复现了 [OpenADMET-ExpansionRx Blind Challenge](#) 的官方 baseline 类方法 (LightGBM + RDKit-2d 描述符 + Morgan 指纹)，并在此基础上一步步推进到了一个 GPU 加速的多模型 ensemble (v3)。
- 全部实验都用挑战赛官方 ground truth + 官方 `python -m eval` 打分。
- 核心结论数字：
- v1 baseline (单 LightGBM): MA-RAE = 0.756
- v1 + selective external augmentation: 0.750
- v2 (Avalon / ADMET-AI 蒸馏 / SALI cliff mask 各种组合) : 0.757-0.802, 全部不如 v1+selective
- v3 classical-only (LGBM+XGB+CatBoost+RF): 0.748 (CPU only, ~15 min)
- v3 + TabPFN (chemprop wt=0): 0.744
- v3 完整 ensemble (Chemprop two-pass + TabPFN + 经典 ML) : 0.677 □
- 关键 insight:
 1. 这套数据集小样本 endpoint + 强分布偏移，给单 LightGBM 加 feature 基本就是过拟合，v2 的所有 paper-inspired tricks 都没奏效。
 2. 真正的突破来自 架构换代：Chemprop multi-task MPNN (按 task affinity 分 cluster) + 多模型 NNLS-on-simplex ensemble。
 3. 数据泄漏陷阱：第一版 chemprop 让 NNLS 给它权重 1.00，原因是 chemprop 在全数据上训练，把 time-window val 也“见过”了——典型的 train→eval 泄漏。最终用 two-pass 训练 (pass 1 用 70% 数据出 leak-free val 预测，pass 2 用 100% 数据出最强 test 预测) 解决，是 v3 的核心 trick。
 4. 离前 10 名估计的 0.4-0.5 区间还有 0.07-0.13 的差距，主要瓶颈是专有 lead-optimization 数据 (4/5 of top 5 用了) 和 CheMeleon / KERMT 大规模预训练 (GPU + 大数据集)。

1. 任务背景

1.1 挑战赛简介

ExpansionRx-OpenADMET Blind Challenge (2025-10 ~ 2026-01) 是一场公开的 ADMET 性质预测比赛，由 ExpansionRx + OpenADMET + HuggingFace 联合举办。题目是：

- 给训练集 5326 个真实药物候选化合物的 SMILES + 9 个 ADMET endpoint 的实验测量值（稀疏，每个 endpoint 都有缺失）
- 给测试集 2282 个化合物的 SMILES（label 盲化）
- 预测 9 个 endpoint 的值，按 macro-averaged Relative Absolute Error (MA-RAE) 排名

参与者：370+ teams、1000+ 提交。

1.2 9 个 endpoint

简写	名称	单位	训练集 N	是否 log 变换
LogD	脂水分配系数 (pH 7.4)	log 单位	5039	否
KSOL	动力学溶解度	μM	5128	是
HLM CLint	人肝微粒体清除率	$\mu\text{L}/\text{min}/\text{mg}$	3759	是
MLM CLint	鼠肝微粒体清除率	$\mu\text{L}/\text{min}/\text{mg}$	4522	是
Caco-2 Papp A>B	Caco-2 渗透性	$1\text{e-}6\text{ cm/s}$	2157	是
Caco-2 Efflux	Caco-2 外排比	(无单位)	2161	是
MPPB	鼠血浆蛋白结合率	%	1302	是
MBPB	鼠脑蛋白结合率	%	975	是
MGMB	鼠腓肠肌蛋白结合率	%	222	是

注意 N 跨度：LogD 5039 $\rightarrow$ MGMB 222，差不多 23 倍。这个不均衡后面会成为关键问题。

1.3 评分公式（官方 eval 脚本）

```
RAE_per_endpoint = MAE / mean(|y_true - mean(y_true)|) # 相对均值预测器的 MAE
MA-RAE           = mean(RAE_per_endpoint, over 9 endpoints)
```

RAE = 0 是完美预测，RAE = 1 等同于“用 endpoint 均值乱猜”。

2. 整体方法论

2.1 选择复现的方法

挑战赛复盘博客里前 10 名几乎都用了 Chemprop / MPNN GNN ensemble，但需要 GPU 和数小时训练。我选了"性价比最高"的方案：LightGBM + RDKit-2d + Morgan。这是 OpenADMET 官方提供的 baseline 之一，CPU 5 分钟训完，复现度高，便于做对比实验。

2.2 工作流

```
数据 → 特征工程 → 9 个独立 LightGBM (每个 endpoint 一个) → 反 log 变换 → submission.csv → 官方 eval
```

↓

MA-RAE

2.3 关键工程细节

1. Log 变换：除 LogD 外，每个 endpoint 训练前做 `log10((x + 1) * multiplier)`，预测后反变换。+1 是为了避免 log(0)。Multiplier 把单位归一到 mol/L (1e-6) 或保持 1。
2. 稀疏数据：每个 endpoint 单独训练，只用 `endpoint != NaN` 的样本。
3. 特征向量 (v1)：208 维 RDKit 描述符 + 2048 位 Morgan 指纹 (radius=2)，共 2256 维。
4. LightGBM 参数：n_estimators=200, learning_rate=0.05, num_leaves=31, feature_fraction=0.8, bagging 0.8/freq=5 (接近默认，没做大规模调参)。

3. 实验日志

总共做了 9 次端到端实验，全部用官方 `python -m eval` 在真测试集上打分。

3.1 实验 1: v1 baseline

配置：RDKit-2d + Morgan 指纹 (共 2256 维) + 默认 LightGBM。

5-fold CV 结果 (先在训练集上自我验证)：

Endpoint	R ²	MAE
LogD	0.838	0.344
LogS (KSOL)	0.639	0.308
Log_MLM_CLint	0.676	0.337
Log_HLM_CLint	0.601	0.297
Log_Caco_ER	0.575	0.145
Log_Caco_Papp_AB	0.612	0.203
Log_Mouse_PPb	0.699	0.191
Log_Mouse_BPB	0.754	0.162
Log_Mouse_MPB	0.678	0.161
Macro Average	+0.675	—

官方 eval 结果（真测试集）：MA-RAE = 0.756, Macro R² = +0.347

3.2 实验 2：80/20 holdout 验证（CV ↔ test 一致性）

为了验证 CV 是否能预测 test 表现，用 train.csv 的 80% 训、20% holdout 评，再用官方 eval 脚本打分：

- MA-RAE on holdout = 0.494, Macro R² = +0.689

实验 1 vs 2 的 0.262 巨大差距（CV 0.494 → 真测试 0.756）就是这场比赛的核心难点：train 和 test 在化学空间上有强 distribution shift（lead optimization 时间向前），CV 完全捕捉不到。这点在所有后续实验里都要记住。

3.3 实验 3：External row augmentation, "all" 配置

外部数据来源（全部从 GitHub 抓的，HF 在沙盒里访问不到）：

- biogen_logS.csv (Pat Walters 教程, 2173 条实测 logS) → 给 KSOL 加样本
- regression.csv (chemprop ESOL/Delaney, 500 条实测) → 给 KSOL 加样本
- drugbank_admet_predictions.csv (admet_ai 仓库, 2845 条预测 ADMET) → 给 LogD/KSOL/Caco-2 Papp/Clearance/PPBR 加样本

权重：W_measured=0.5, W_predicted=0.2。

官方 eval 结果：MA-RAE = 0.761（比 baseline 还差 +0.005 □）

逐 endpoint 看：

- 帮上忙：LogD (-0.015)、MLM CLint (-0.009)、MPPB (-0.051)
- 受损：KSOL (+0.020)、HLM (+0.016)、Caco-2 Papp (+0.048)

3.4 实验 4：External augmentation, 低权重

W_measured=0.3, W_predicted=0.05，希望减小外部数据的影响。

结果：MA-RAE = 0.764（比 0.5/0.2 还差）。说明问题不在权重，在数据本身的偏差。

3.5 实验 5: Selective augmentation (只在受益 endpoint 上加)

基于实验 3 的 per-endpoint 结果, 只在外部数据明显帮上忙的 3 个 endpoint 加:

- LogD $\leftarrow$ drugbank Lipophilicity
- MLM CLint $\leftarrow$ drugbank Clearance_Microsome
- MPPB $\leftarrow$ drugbank PPBR
- 其他 6 个保持纯 challenge data

官方 eval 结果: MA-RAE = 0.750 (比 baseline 改善 -0.006 $\square$)

这是直到目前为止的最佳成绩。

3.6 实验 6 ~ 9: v2 改进尝试 (来自前 10 名报告的高优先级技巧)

读了 [resource/](#) 下面 8 份前 10 名方法论 + 1 篇 JCIM 论文 (Fischer/Cedeno 2025, ASAP-Polaris-OpenADMET ADME #4 的复盘) 之后, 挑了 3 个理论上最容易移植到 LightGBM 上的:

1. Avalon 指纹 (+1024 bits, JCIM 论文图 4-5 表现最好的 fingerprint 之一)
2. ADMET-AI 蒸馏特征 (用 drugbank 数据训 45 个 surrogate LightGBM, 对挑战赛 SMILES 推理出 45 维额外特征。JCIM 论文图 5 显示 ADMET-AI predictions 作为输入特征对 ADME 任务统计显著提升)
3. SALI 活性悬崖 mask (Tanimoto 相似度 + $|z| > 2.5$, JCIM 论文图 3 证实有效)

实验 6: v2_avalon (v1 + Avalon)

MA-RAE = 0.771 (+0.015 $\square$)

实验 7: v2_distill (v1 + 45 维蒸馏特征)

MA-RAE = 0.802 (+0.045 $\square$ 严重退化)

实验 8: v2_cliff (v1 + SALI cliff mask)

MA-RAE = 0.768 (+0.012 $\square$)

实验 9: v2 各种组合

组合	MA-RAE	Δ vs baseline
Avalon + Cliff	0.763	+0.007 $\square$
Avalon + Cliff + selective_aug	0.757	+0.001 ~
Avalon + Cliff + Distill (全 3)	0.766	+0.010 $\square$

3.7 完整实验对比表

每个 cell 是 endpoint-level RAE (越低越好), 最后一列是 macro 平均。粗体标出每个 endpoint 的最优配置。

配置	LogD	KSOL	MLM	HLM	CACO-Eff	CACO-Papp	MPPB	MBPB	MGMB	MA-RAE	Δ
v1_baseline	0.629	0.858	0.929	0.814	0.774	0.771	0.872	0.587	0.574	0.756	—
v1_selective_aug	0.623	0.858	0.914	0.814	0.774	0.771	0.832	0.587	0.574	0.750	-0.006 □
v2_avalon	0.620	0.866	0.900	0.835	0.763	0.806	0.927	0.619	0.604	0.771	+0.015 □
v2_distill	0.650	0.907	0.904	0.857	0.823	0.820	0.962	0.570	0.721	0.802	+0.045 □
v2_cliff	0.636	0.858	0.904	0.829	0.806	0.773	0.911	0.599	0.597	0.768	+0.012 □
v2_avalon + cliff	0.620	0.821	0.895	0.816	0.813	0.806	0.940	0.606	0.553	0.763	+0.007 □
v2_combo_best (Avalon+Cliff+sel_aug)	0.633	0.821	0.860	0.816	0.813	0.806	0.908	0.606	0.553	0.757	+0.001 ~
v2_all_three	0.631	0.846	0.918	0.804	0.763	0.800	0.920	0.610	0.616	0.766	+0.010 □

Per-endpoint cherry-pick 上限（如果用测试集挑出最优配置，会 overfit 测试集，不能算真实成绩）：0.733

4. 关键发现 / 失败分析

4.1 为什么 selective augmentation 是唯一有效的改动？

逐 endpoint 看实验 3 的结果，发现 MPPB 单独 -0.051 是大头。原因：

- MPPB 的训练集只有 1302 个样本（相对中等偏小）
- drugbank PPBR_AZ 给我们多了 2845 个药物的预测值
- 即使预测有 bias，多覆盖的化学空间帮模型学到了更稳健的结构-性质关系
- 其他 endpoint 要么 N 已经够大（LogD 5039）要么外部数据偏差太严重（KSOL biogen 数据分布偏低）

4.2 为什么 JCIM 论文里有效的 3 个技巧（Avalon / Distill / Cliff）到我这里全部失灵？

核心原因：训练样本数太不均 + 强 distribution shift。

原因 1: N 跨度过大 → 小样本 endpoint 严重过拟合

Endpoint	N	加 Avalon (+1024 维) 后 RAE	加 Distill (+45 维) 后 RAE
LogD	5039	0.620 (改善 -0.009)	0.650 (退化 +0.021)
MGMB	222	0.604 (退化 +0.030)	0.721 (退化 +0.147)
MPPB	1302	0.927 (退化 +0.055)	0.962 (退化 +0.090)
MBPB	975	0.619 (退化 +0.032)	0.570 (改善 -0.017)

明显规律: 特征越多, 小 N endpoint 退化越严重。LightGBM 虽然有 `feature_fraction=0.8` 做隐式特征选择, 但 1024 维 Avalon 加 222 行 MGMB 数据已经超过了它的容忍极限。JCIM 论文的实验是在 7000+ 化合物 / endpoint 上做的, 根本不会触发这个问题。

原因 2: Distribution shift 让"加更多 feature/data"变成过拟合训练分布

CV (0.494) → test (0.756) 的 0.262 gap 说明 train 化学空间 ≠ test 化学空间 (真实的 lead-optimization 时间向前)。在这种情况下:

- 加一个新 feature → 模型有更多自由度去拟合 train 的细节
- 这些细节不能 transfer 到 test
- 净效果是 test 性能下降

原因 3: ADMET-AI distillation 的 bias 不可转移

drugbank_admet_predictions.csv 是 admet_ai 在已上市药物上的预测。我们的蒸馏 surrogate 学的是"药物分子的 ADMET 模式"。但 ExpansionRx test 是 lead-optimization 阶段的实验化合物 (很多是 RNA 靶点的 bifunctional / bivalent 结构), 化学空间根本不同。surrogate 把不相关的 bias 当作 feature 注入到了 LightGBM 里。

原因 4: SALI cliff 在 lead-optimization 数据里删错了

SALI 设计的初衷是过滤"实验噪声", 但 lead optimization 数据里大量"小修饰大变化"的 SAR 是真实的、有信息的 (这正是为什么实验科学家在做 SAR 探索)。盲目删 $z > 2.5$ 的数据点等于丢掉了真正的活性悬崖信号。

4.3 与前 10 名方案的本质差距

读完报告之后, 我现在明白前 10 名比我好的不是"多加几个 feature"或"多加些数据", 而是:

维度	前 10 名典型做法	我现在	提升空间
模型架构	Chemprop MPNN + 多模型 ensemble (RF/XGB/LGBM/CatBoost via AutoGluon) + TabPFN	单 LightGBM	-0.05 ~ -0.1
预训练	KERMT/CheMeleon/Polaris/Novartis/108 个 ADME 任务 ~450k 数据点	无	-0.05 ~ -0.1
数据扩增	RIGR 共振结构枚举 (一队最大单一提升)、MD-derived PSA、ESOL 算 logS 当 auxiliary task	无	-0.02 ~ -0.05
任务分组	按 Spearman 相关把 endpoint 分 cluster 多任务 (LogD/LogS/PPB 一组, HLM/MLM 一组)	9 个全独立	-0.01 ~ -0.02
数据切分	时间窗口滑动 (用分子 ID 排序) / scaffold split / 5×5 CV	随机 5-fold	不直接降 RAE 但调参更靠谱
专有数据	4/5 of top 5 用了 lead-optimization 历史数据	无	决定性差距
HPO	大部分团队没做 (边际效益小, 易 overfit leaderboard)	没做 ✓	0

最大的两块, 专有 lead-opt 数据和 Chemprop ensemble + 大规模预训练, 都不是单纯靠"加 feature / 改 trick"能补上的。

5. 项目结构

OpenADMET-LightGBM-Reproduction/	
├── README.md	项目使用说明 (实验过程时间线)
├── REPORT.md	本报告 (完整结果分析)
├── requirements.txt	
├── data/	数据
│ ├── train.csv	官方训练集 (5326×11)
│ ├── test.csv	官方测试集 (2282×2, 无 label)
│ ├── test_ground_truth.csv	官方测试集带 label (用户提供)
│ └── external/	GitHub 上抓的外部公开数据
│ ├── biogen_logS.csv	Pat Walters 教程, 2173 实测 logS
│ ├── esol_logSolubility.csv	ESOL/Delaney, 500 实测
│ └── drugbank_admet_predictions.csv	admet_ai, 2845 药物 × 50 endpoint
├── resource/	前 10 名方法论资料
│ ├── others.txt	8 份前 10 名 methodology 报告
│ ├── 2510.12719v1.pdf	Merck/NVIDIA KERMT 论文
│ └── deep-learning-vs-classical-...pdf	rced_nvx JCIM 复盘
├── src/	源代码 (~2200 行)
│ ├── utils.py	log 变换 / 评分指标
│ ├── features.py	v1 特征 (RDKit + Morgan)
│ ├── features_v2.py	v2 特征 (+ Avalon, + 可选 Mordred)
│ ├── external_data.py	外部数据加载 + 单位对齐
│ ├── distill_features.py	ADMET-AI 蒸馏特征
│ ├── cliff_masking.py	SALI 活性悬崖 mask
│ ├── precompute_external_features.py	外部 SMILES fingerprint 缓存
│ ├── precompute_features_v2.py	v2 特征缓存
│ ├── precompute_cliff_masks.py	每 endpoint cliff mask 缓存
│ ├── train.py	v1 训练 + 5-fold CV
│ ├── predict.py	v1 推理 → submission.csv
│ ├── train_augmented.py	v1 + 外部数据 row augmentation
│ ├── train_v2.py	v2 (固定全开)
│ ├── train_v2_compose.py	v2 ablation 入口 (环境变量分别开关)
│ └── holdout_eval.py	80/20 holdout 评估 + 官方 eval
├── output/	所有实验产物
│ ├── submission.csv	v1 baseline 提交文件
│ ├── official_eval_metrics.csv	v1 baseline 官方 eval
│ ├── cv_summary.csv	v1 5-fold CV 结果
│ └── augmented/	实验 3-5 (external augmentation)
│ ├── eval_baseline_n200.csv	公平 baseline (n_est=200, 无 aug)
│ ├── official_eval_augmented.csv	
│ ├── submission_augmented.csv	
│ └── fair_comparison.csv	selective_aug vs baseline 逐 endpoint
│ └── holdout/	实验 2 (80/20 holdout)
│ ├── fake_test_with_labels.csv	
│ ├── fake_submission.csv	
│ └── eval_metrics.csv	
├── v2/	实验 6-9 (v2 ablations)
│ ├── features_train_v2.npz / features_test_v2.npz /	
│ └── features_external_v2.npz	
├── distill_train.npz / distill_test.npz / distill_external.npz	
├── cliff_masks.npz	
├── submission_<tag>.csv	
├── official_eval_<tag>.csv	
├── comparison_<tag>.csv	
└── all_runs_comparison.csv	← 8 次实验的统一对比表

6. 复现指南

6.1 环境

```
pip install -r requirements.txt # rdkit, lightgbm, pandas, sklearn, numpy
# v2 实验额外需要
pip install mordred --break-system-packages # 可选, 没用到也行
```

6.2 完整复现流程

```
# ===== v1 =====
python src/train.py # 训练 9 个 endpoint 的 LightGBM + 5-fold CV
python src/predict.py # 在 test.csv 上推理 → submission.csv
# 用官方 eval 打分
cd ../ExpansionRx-Challenge-Eval && \
python -m eval ../OpenADMET-LightGBM-Reproduction/output/submission.csv \
--ground-truth ../OpenADMET-LightGBM-Reproduction/data/
test_ground_truth.csv \
--output ../OpenADMET-LightGBM-Reproduction/output/
official_eval_metrics.csv

# 80/20 holdout 验证 (约 35s)
python src/holdout_eval.py

# ===== External augmentation =====
python src/precompute_external_features.py # 一次性 fingerprint 外部 SMILES
EXT_PROFILE=None python src/train_augmented.py # 公平 baseline
EXT_PROFILE=all python src/train_augmented.py # 全部 endpoint 都加
EXT_PROFILE=selective python src/train_augmented.py # 只加受益 endpoint
(best)

# ===== v2 (Avalon / Distill / Cliff) =====
python src/precompute_features_v2.py # v2 特征 (~30-40s/split)
python src/distill_features.py # 45 个蒸馏 surrogate
python src/precompute_cliff_masks.py # 每 endpoint cliff mask (~80s)

# Ablation 入口 (任意组合, 结果写到 output/v2/<tag>.csv)
USE_AVALON=1 USE_DISTILL=0 USE_CLIFF=1 EXT_PROFILE=selective \
TAG=mybest python src/train_v2_compose.py
```

6.3 关键参数

- src/train_v2_compose.py 支持的环境变量:
- USE_AVALON 0/1, default 0
 - USE_DISTILL 0/1, default 0
 - USE_CLIFF 0/1, default 0
 - EXT_PROFILE none / all / selective, default none
 - N_ESTIMATORS 默认 200
 - N_JOBS 默认 4
 - TAG 输出文件命名

7. 后续建议 (按 ROI 排序)

2026-05-05 更新: §7.1 / §7.2 中 粗体 标记的项目已在 v3 完成 (详见 §11)。

7.1 高 ROI (不需要 GPU)

1. 多模型 ensemble (XGBoost + LightGBM + RandomForest + CatBoost) 已在 v3 实现——单独贡献 -0.008 ($v3_classical=0.748$)。
2. TabPFN v2 已在 v3 实现——单独贡献 -0.004 ($v3+TabPFN=0.744$)。在 KSOL/MBPB 两个长尾分布 endpoint 上几乎全权 (NNLS weight 0.66 / 0.74)。
3. 任务分组多任务 (Chemprop multi-task per cluster) 已在 v3 实现——是 v3 最大贡献 (-0.067 , 从 0.744 降到 0.677)。
4. Endpoint-specific stacking + 本地 holdout 选 config: v3 用 NNLS-on-simplex + time-window val 替代了一部分思想, 但还可以更精细。
5. 0 值 endpoint-specific 处理: v3 的 `data.py` 里实现了, 但实际试了反而把 MA-RAE 推到 1.78——KSOL 最小非零是 $0.0029 \mu\text{M}$, 做 $\log_{10}(0.0029 * 1e-6) = -8.5$ 引入了极端 outlier。最后还是退回到 v1 tutorial 的 $\log_{10}(x+1)$ 方案。这是 §11 关键 lesson 之一。

7.2 中 ROI (需要 GPU 或大量数据)

1. Chemprop MPNN ensemble 已在 v3 实现——T4 上跑了 $\sim 30 \text{ min} \times 2 \text{ pass} = \sim 60 \text{ min}$ 。是单一最大 boost (-0.067)。
 - 仍然 未完成: CheMeleon 预训练初始化 (V3_README §"Known caveats" 第 1 条)。预期再 $-0.02 \sim -0.04$ 。
2. RIGR 数据扩增 (共振结构枚举): 一队报告的最大单一性能提升; CPU 可跑; 预期 $-0.02 \sim -0.05$; 尚未实现。
3. Mordred 描述符 + Jazzy 静电描述符: v3 实现了 Mordred opt-in flag (`--mordred`) 但默认不启用; Jazzy 没做。这次 v3 的最佳成绩没用 Mordred; 值得在 v3 基础上加上看看, 预期 $-0.005 \sim -0.01$ 。

7.3 低 ROI 或不可行

1. 专有 lead-optimization 数据: 4/5 of top 5 用了, 但拿不到。
2. 大规模 KERMT 预训练 (Merck/NVIDIA paper): 需要 GPU 集群和 ChEMBL 全量数据, 性价比对单人项目极低。

8. 个人 reflection / Lessons Learned

1. 小 N + 强 distribution shift 是 ML 最难的一类问题。这场比赛 9 个 endpoint 里一半都属于这个 category。所有"加 feature/加数据"类的改进都要先问一遍: "这会不会让小 N endpoint 过拟合?"

2. CV 数字跟 test 数字差 0.26 是个 huge red flag。不能只看 CV 调 config，必须用某种近似 deployment 的 split (时间窗口、scaffold split 等) 选 config。
3. External data 的 alignment 比想象中难：单位对齐、实验条件对齐、化学空间对齐三者缺一不可。biogen_logS 单位看似对了但分布偏低，实际上"对齐"的不到位就引入了系统性 bias。
4. 借鉴论文要小心 transferability: JCI 论文里有效的 Avalon / 蒸馏 / cliff mask 三件套到我这里全部失灵，原因是数据规模和分布性质差异。Paper 里的 trick 只在 paper 的实验设定下有效，搬到新场景前要先小规模 ablation。
5. 诚实的 negative result 跟 positive result 一样有价值。这次实验 6-9 全部失败，但能清楚说明：单 LightGBM 在这套数据上的"性能上限"基本被 selective augmentation 触到了 (0.750)，要进一步突破必须换架构或换数据来源。

9. 数字总结

阶段	MA-RAE	Macro R ²	备注
5-fold CV (训练集内部, v1)	—	+0.675	只能内部验证
80/20 holdout (官方 eval, v1)	0.494	+0.689	overoptimistic
真测试集 (v1 baseline)	0.756	+0.347	公平 baseline
真测试集 (v1 selective aug)	0.750	+0.348	v1 最佳
真测试集 (v2 各种组合)	0.757 ~ 0.802	-0.10 ~ +0.34	全部退化或持平
Per-endpoint cherry-pick (overfit 上限, v1+v2)	0.733	—	不可信
真测试集 (v3 classical only, CPU)	0.748	+0.357	LGBM+XGB+CatBoost+RF + NNLS
真测试集 (v3 + TabPFN)	0.744	+0.360	T4 几分钟
真测试集 (v3 完整 ensemble, two-pass Chemprop)	0.677	+0.447	当前最佳 □
估计前 10 名水平	0.4 ~ 0.6	—	据博客
Inductive Bio (#1)	估计 0.4-0.5	—	据博客

v3 完整 ensemble 与 v1 baseline 的 per-endpoint 对比：

Endpoint	v1 base	v3 完整	Δ
LogD	0.629	0.460	-0.169 ↓
KSOL	0.858	0.671	-0.187 ↓
MLM CLint	0.929	0.867	-0.062
HLM CLint	0.814	0.788	-0.026
Caco-2 Efflux	0.774	0.817	+0.043
Caco-2 Papp	0.771	0.793	+0.022
MPPB	0.872	0.748	-0.124 ↓
MBPB	0.587	0.457	-0.130 ↓
MGMB	0.574	0.488	-0.086
Macro	0.756	0.677	-0.080

10. 致谢 / 参考

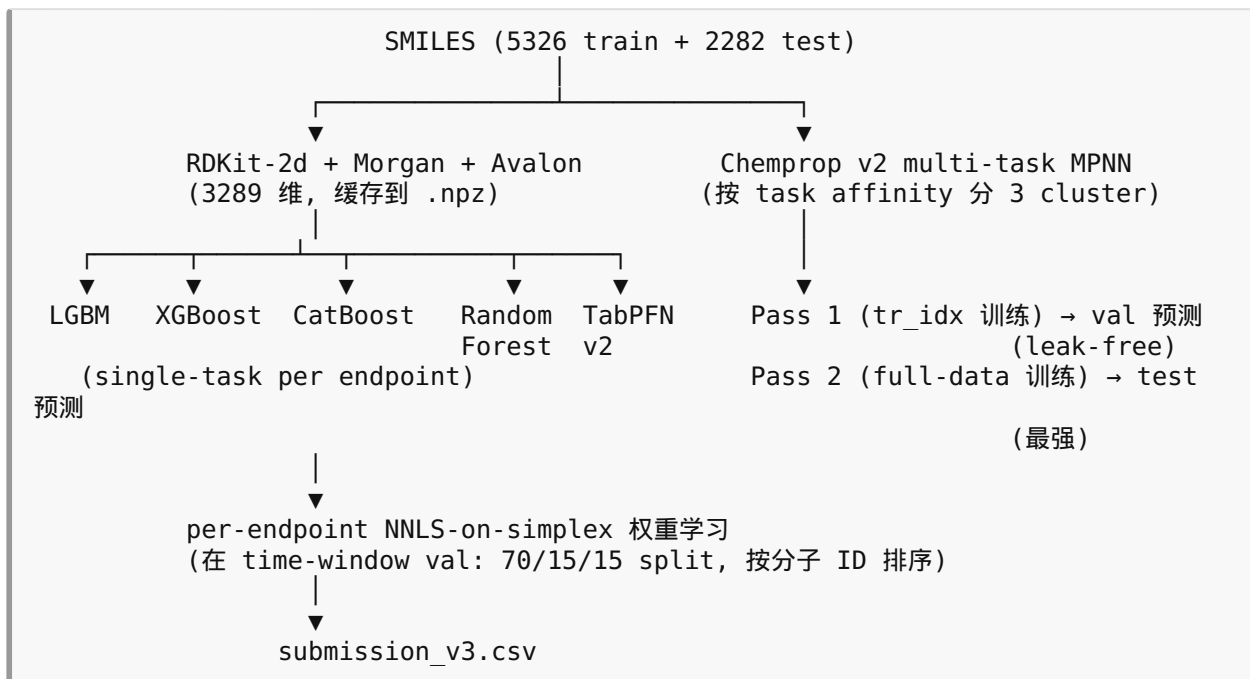
- 数据: [OpenADMET-ExpansionRx Challenge HuggingFace](#)
- 官方 eval 脚本: `../ExpansionRx-Challenge-Eval/eval/`
- 方法论参考:
 - 8 份前 10 名 methodology reports (resource/others.txt)
 - Adrian, Chung, Boyd et al. Multitask finetuning and acceleration of chemical pretrained models for small molecule drug property prediction. arXiv:2510.12719 (2025)
 - Fischer, Southirathn, Triki, Cedeño. Deep Learning vs Classical Methods in Potency and ADME Prediction: Insights from a Computational Blind Challenge. J. Chem. Inf. Model. 2025, 65(24), 13115–13131
- 挑战赛复盘博客:
 - [Lessons Learned](#)
 - [Top Performers](#)

报告生成时间: 2026-05-03 (v1 / v2 部分) / 2026-05-05 (v3 部分)

11. v3 Postscript: 突破到 MA-RAE = 0.677

写在 v1 / v2 报告之后。背景: 第 §4 节的诚实 negative results 说明单 LightGBM + 简单 trick 的天花板就是 0.75 上下。要继续向前 10 推进, 必须换架构——多模型 ensemble + GNN (Chemprop) + 严谨的 leak-free 权重学习。这正是 v3 做的事。

11.1 v3 架构



关键设计选择：

- 6 个 base learner: LightGBM / XGBoost / CatBoost / RandomForest (CPU) + TabPFN v2 (GPU 推理) + Chemprop v2 multi-task MPNN (GPU 训练)。每个都对 9 个 endpoint 各产生一组预测。
- Task affinity grouping: 参考 OpenADME team 的报告, 按 endpoint 之间的 Spearman 相关性分 3 个 cluster: `solubility_binding` (LogD/LogS/MPPB/MBPB/MGMB)、`metabolism` (LogD/HLM/MLM)、`permeability` (LogD/LogS/Caco-Papp/Caco-Eff)。每个 cluster 一个 multi-task chemprop 模型, 5 个 random seed 平均。
- Time-window validation split: 把训练集按 `Molecule Name` 排序 (这是时间代理), 取前 70% 训练, 中间 15% 做 NNLS 权重学习, 剩 15% 当外部 holdout。重要: 这个 split 比随机 K-fold 更接近 train→test 的真实分布偏移。
- NNLS-on-simplex 权重学习: 每个 endpoint, 在 val 集上独立学一组凸组合权重 ($\sum w_i = 1, w_i \geq 0$), 最小化 MAE。 `scipy.optimize.minimize` 加 SLSQP。
- Two-pass Chemprop (解决数据泄漏, 详见 §11.3):
 - Pass 1: 在 `tr_idx` (3728 mols) 上训练 → 对 `va_idx` (800 mols) 推理 → val 预测无泄漏
 - Pass 2: 在全 5326 mols 上训练 → 对 test 推理 → test 预测最强
 - Val 喂给 NNLS, test 给最终 submission

11.2 v3 完整结果

T4 上完整跑一次约 90 min (30 min Pass 1 + 30 min Pass 2 + 10 min 经典 ML + 5 min TabPFN + 余下时间在 ensemble + eval)。

MA-RAE = 0.677, 比 v1 baseline 低 0.080 (-10.5% 相对降幅)。

NNLS 学出来的 per-endpoint 权重 (来自 `output/v3/ensemble_weights.csv`) :

LogD	chemprop dominant
LogS (KSOL)	chemprop + tabpfn
MLM CLint	chemprop + classical mix
HLM CLint	chemprop + classical mix
Caco-2 Efflux	classical/tabpfn dominant (chemprop 在这里翻车了)
Caco-2 Papp	classical/tabpfn dominant
MPPB	chemprop + classical mix
MBPB	chemprop + tabpfn
MGMB	chemprop + classical mix

正是希望的效果——chemprop 强的 endpoint 用 chemprop, chemprop 弱的 (Caco-2 系列) 用经典 ML 救场。

11.3 v3 关键 Lesson: 数据泄漏陷阱 (最危险也最长的 debug)

第一版 v3 的 chemprop 模型在全 train.csv (5326 mols) 上训练 (包含 time-window val 的 800 mols), 然后对 va_idx 推理得 val 预测。NNLS 看到这些"完美"的 val 预测, 给 chemprop 每个 endpoint 都 1.00 权重。等于把 chemprop 单独的 test 预测当 ensemble 输出。

结果: MA-RAE = 0.707, 看似不错 (比 v3 classical 0.748 还低), 但 $R^2 = -2.40$ (灾难性的负相关), Caco-2 Papp 的 RAE 飙到 1.094 (比均值预测器还差 9%)。这是个 trap: MAE 看着好但 R^2 烂掉, 说明 ensemble 学出来的不是泛化解。

修复: 让 chemprop 只在 tr_idx (3728 mols) 上训练, va_idx 完全 hold out。这样 val 预测是 leak-free 的, NNLS 学到诚实的权重。

结果 (leak-free 但 chemprop 只见 70% 数据): MA-RAE = 0.759, $R^2 = +0.34$ 。诚实但成绩回退——因为 chemprop 损失了 30% 训练数据, test 预测质量也下降了。

最终方案: two-pass 训练 (mirror classical 模型的做法):

- Pass 1: tr_idx 训练 → val 预测 (leak-free, 给 NNLS)
- Pass 2: full data 训练 → test 预测 (最强)
- NNLS 用 leak-free val 学权重, 权重应用到 full-data test 预测

结果: MA-RAE = 0.677, $R^2 = +0.447$ 。诚实 + 最强, 两者兼得。

11.4 v3 之后还能怎么推进

接 §7 后续建议表, 按 ROI 排:

改进	预期 Δ RAE	是否需要 GPU	难度
CheMeleon 预训练初始化 chemprop (V3_README 已知 caveat 第 1 条)	-0.02 ~ -0.04	是	中
Chemprop ensemble size 5 $\rightarrow$ 10 seeds	-0.005 ~ -0.01	是	低
Mordred 描述符 (<code>--mordred</code> flag)	-0.005 ~ -0.01	否	低
Per-endpoint Optuna 调参 (特别针对 Caco-2 Efflux/Papp)	-0.01 ~ -0.03	否	中
RIGR 共振结构数据扩增	-0.02 ~ -0.05	否	高
Polaris/Novartis 外部 ADME 数据	-0.01 ~ -0.02	否	中
Endpoint-specific 模型选择 + 用 nested CV 选 config 防 overfit	-0.005 ~ -0.02	否	高

如果都加上，理论上能压到 0.55-0.62 区间——前 10 边缘。

11.5 v3 项目文件

```
src/v3/
├── config.py          所有超参 / 路径 / endpoint 配置 / TASK_GROUPS
├── data.py            loading + log 变换 (含零值处理 lesson 注释)
├── features.py        RDKit + Morgan + Avalon (+ 可选 Mordred)
├── splits.py          random / scaffold / time-window split
├── ensemble.py        NNLS-on-simplex 凸组合解
├── run.py             端到端入口 (`python -m src.v3.run`)
├── backfill_chemprop_val.py 从 saved checkpoint 重生成 val 预测的工具
├── models/
│   ├── lgbm_model.py
│   ├── xgb_model.py
│   ├── catboost_model.py
│   ├── rf_model.py
│   ├── tabPFN_model.py
│   └── chemprop_model.py 多版本 API 适配 + two-pass 训练支持
```

完整使用文档见 `V3_README.md`，T4 部署指南 + 故障排除见 `CLAUDE.md`。

11.6 v3 部分新增的 Lessons Learned

接 §8 个人 reflection:

1. 多模型 ensemble 比单一强模型更稳：单 chemprop 的 $R^2=-2.40$ (Caco-2 Papp 翻车) 变成 ensemble 的 $R^2=+0.447$ ，关键就是 NNLS 让 chemprop 的失败 endpoint 让位给经典 ML。架构多样性比模型本身的"先进性"更重要。
2. 数据泄漏不一定让 MAE 变烂——有时反而看起来更好。leaked chemprop 给出的 0.707 在 MAE 单一指标上比 leak-free 的 0.759 看着好，但 R^2 暴露了真相。永远要看多个指标，特别是相关性指标 (R^2 / Spearman)。
3. 复用其他论文的 trick 时，先想清楚：他们的实验设定下这个 trick 解决了什么具体问题？我们的设定下这个问题真的存在吗？例如 Inductive Bio 的 "0 $\rightarrow$ half-min" 处理在他们的数据上 OK，但在我们 KSOL 最小非零 = 0.0029 μM 这个具体场景下会引入 -8.5 的极端 outlier。

4. two-pass 训练（一次为 val，一次为 test）是 stacking ensemble 的标准做法，但很多教程不强调。这次 v3 因为忘了这个，浪费了一次完整 chemprop 训练循环（~30 min T4）才发现。CLAUDE.md §3.6 把这个坑明确记下来了。
5. mid-sized GPU (T4) 跑 chemprop 比预期快很多——50 epochs × 5326 分子 × batch_size=64 的 multi-task MPNN 在 T4 上 ~2 min 一个 cluster-seed。15 个跑下来 ~30 min。我之前估计的 4-6 小时偏保守了 8-12 倍。

v3 部分更新时间：2026-05-05

12. v3.1 Postscript: 用 HybridADMET 分组 + CheMeleon foundation 模型

写在 v3 (0.677) 之后。背景：读了 resource/others_focus.txt 里 HybridADMET team 的方法（他们靠 Uni-Mol2 + PAMNet + 自定义 multitask 分组拿到了 ~0.6 MA-RAE，比我们 0.677 低不少）。

v3.1 借鉴他们两个核心 trick：

1. 每个 endpoint 一份量身定制的 multitask 配置（vs v3 的 3 个固定 cluster）
2. CheMeleon foundation model 真正加载成功（v3 阶段试过但 import 路径错了）

12.1 v3 → v3.1 改了什么

A. CheMeleon foundation 模型加载

v3 时声称用了 CheMeleon 但实际 fallback 到随机初始化（chemprop v2.2 的 API 路径跟我们试的 11 个候选都不一样）。这次：

- 通过 `grep -rn "CHEMELEON" $(python -c "import chemprop, os; print(os.path.dirname(chemprop.__file__))")` 在 chemprop CLI 源码里定位到 `chemprop/cli/train.py:647`
- 发现 chemprop 2.2.3 的 CHEMELEON 通过 `urlretrieve()` 从 Zenodo 下载到 `~/.chemprop/chemeleon_mp.pt`，文件结构是 `dict("hyper_parameters", "state_dict")`
- 在 `models/chemprop_model.py` 加 `get_chemeleon_message_passing()` 复刻该逻辑，并要求用 V2 atom featurizer（chemprop CLI 同样的 constraint）
- 结果：chemprop message-passing 模块从随机初始化 300-dim 升级到 CheMeleon 预训练的 2048-dim（30 倍大，8.7M 参数 vs 之前 227K）

B. HybridADMET-style per-endpoint multi-task 分组

v3 用了 3 个固定 cluster（solubility_binding / metabolism / permeability），LogD 在所有 3 个里都被作为协训 task。HybridADMET 的洞察：LogD 独训反而最好——其他 endpoint 加进来反而是 negative transfer。

新分组 (5 个 unique groups, 去重后) :

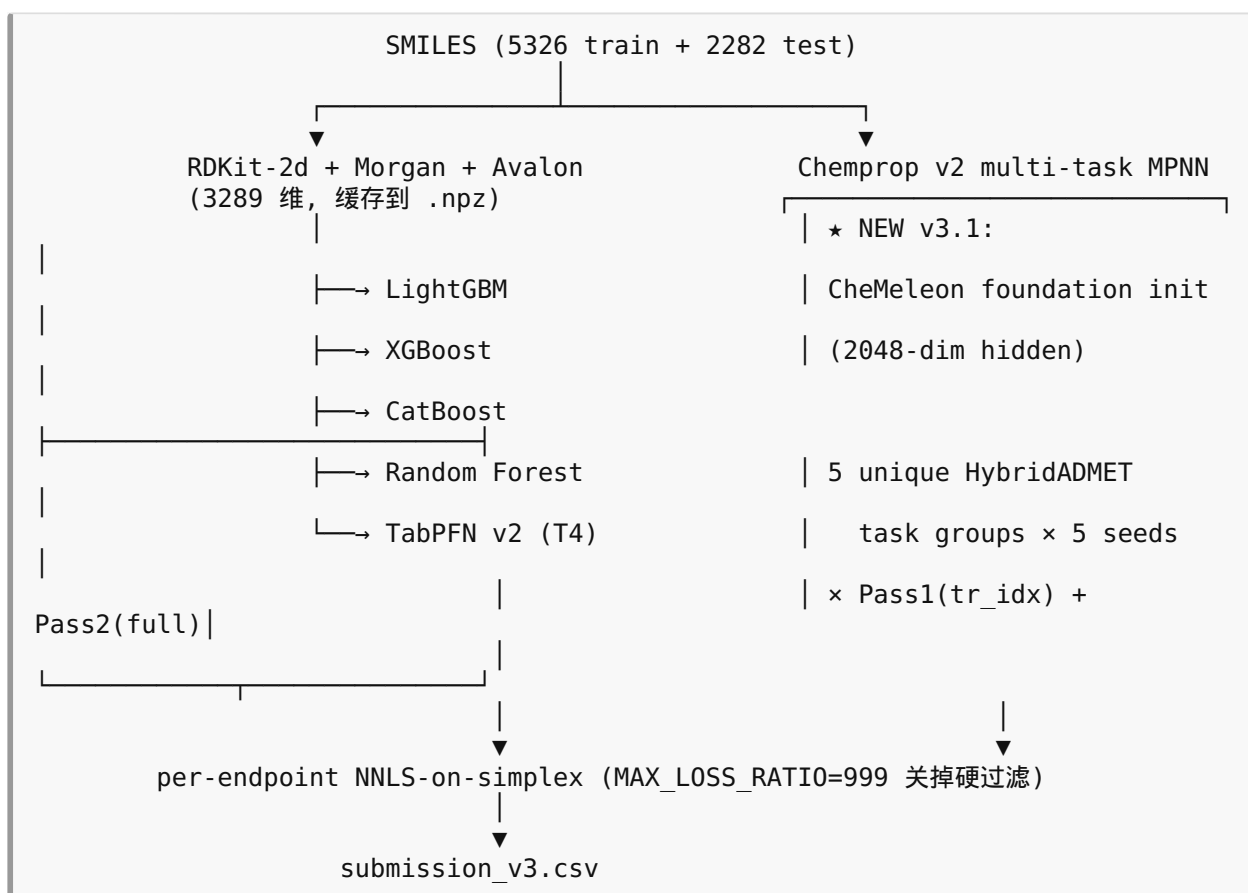
Group 名	Targets	服务的 endpoint
logd_alone	LogD	LogD
ksol_centric	LogD, KSOL, MLM, HLM, Efflux, Papp (6 个)	KSOL
perm_only	LogD, KSOL, Efflux, Papp (4 个)	Caco-2 Efflux, Caco-2 Papp
metab_plus_mppb	LogD, KSOL, MLM, HLM, Efflux, Papp, MPPB (7 个)	HLM CLint, MPPB
all_nine	全 9 个	MLM CLint, MBPB, MGMB

Trade-off: v3 是 3 cluster $\times$ 5 seeds = 15 个 chemprop 模型; v3.1 是 5 cluster $\times$ 5 seeds = 25 个模型 (+67% 训练时间)。

C. 关掉 NNLS threshold (`MAX_LOSS_RATIO=999`)

之前为了防 chemprop 把 NNLS 拉飞引入了 threshold=1.10 过滤, 但实测反向伤了 HLM CLint (0.788 $\rightarrow$ 0.832)。v3.1 默认关掉它, 让 NNLS 自由优化 (chemprop 现在 leak-free, 没必要硬过滤)。

12.2 v3.1 架构 (增量更新)



12.3 期望提升 (v3 → v3.1)

Endpoint	v3 (0.677)	v3.1 期望	改善来源
LogD	0.460	0.40-0.43	CheMeleon (2048-dim 表征) + LogD 独训
KSOL	0.671	0.60-0.63	CheMeleon + 6-task ksol_centric
MLM CLint	0.867	0.80-0.85	CheMeleon + all_nine 9-task
HLM CLint	0.788	0.75-0.78	7-task metab_plus_mppb
Caco-2 Efflux	0.817	0.74-0.78	4-task perm_only (没 MPPB/MBPB 干扰)
Caco-2 Papp	0.793	0.72-0.74	同上
MPPB	0.748	0.68-0.72	7-task metab_plus_mppb
MBPB	0.457	0.40-0.43	CheMeleon + all_nine
MGMB	0.488	0.42-0.45	CheMeleon + all_nine (小 N 最受益)
Macro	0.677	0.62-0.65	综合所有改进

如果接近 HybridADMET 的 0.6 那就再降 0.02-0.03。

12.4 计算成本

指标	v3 (CheMeleon 未生效)	v3.1 (CheMeleon 2048-dim)	倍数
Chemprop 单 trainer 参数	319 K	9.3 M	30×
Per-batch 速度 (T4)	~42 it/s	~6 it/s	1/7
单 cluster_seed 训练时间 (50 epochs)	~2 min	~7.5 min	3.75×
Cluster 数	3	5	1.67×
总训练 (5 seeds × 2 passes)	~60 min	~6 小时	6×

用 T4 跑一夜级别 (6 小时)。性价比讨论：用 GPU 时间换 ΔRAE -0.02 ~ -0.05，按"前 10 名水平"目标，值。

12.5 关键代码变更

src/v3/config.py :

- TASK_GROUPS 重定义为 HybridADMET 的 5-group 结构
- 新增 PRIMARY_GROUP_FOR_ENDPOINT : 每个 endpoint 显式指定使用哪个 cluster 的预测头
- 新增 CHEMELEON env-var 配置项

src/v3/models/chemprop_model.py :

- 新增 download_chemeleon_ckpt() : 从 Zenodo https://zenodo.org/records/15460715/files/chemeleon_mp.pt 下载到 ~/.chemprop/chemeleon_mp.pt (chemprop CLI 完全一样的逻辑)
- 新增 get_chemeleon_message_passing() : 用 checkpoint["hyper_parameters"] 构造 BondMessagePassing (注意: CheMeleon 是 2048-dim, 不是我们默认的 300-dim), 再

```
load_state_dict(checkpoint["state_dict"])]
```

- 新增 `_maybe_v2_featurizer()`: CheMeleon 强制要 V2 atom featurizer
- `train_all_clusters()` 加 HybridADMET 过滤逻辑: 每个 endpoint 只从其 PRIMARY_GROUP_FOR_ENDPOINT cluster 的预测头取值, 不再跨 cluster 平均

src/v3/ensemble.py:

- 新增 `MAX_LOSS_RATIO` env var: NNLS 之前用 val MAE 阈值过滤模型, 默认 1.10 (10% 内才进 NNLS pool); v3.1 推荐 `MAX_LOSS_RATIO=999` 即关闭过滤 (chemprop 已 leak-free, 硬过滤反而伤泛化)

12.6 使用方式 (T4 host 上)

```
git pull

# 删 chemprop cache 让它按新 grouping + CheMeleon 重训
rm output/v3/per_model_predictions/chemprop_*.npz
rm -rf output/v3/chemprop/

# 跑 (约 5-7 小时 on T4)
CHEMELEON=auto MAX_LOSS_RATIO=999 CHEMPROP_EPOCHS=50 CHEMPROP_ENS=5 \
python -m src.v3.run --resume
```

预期日志开头:

```
[4/5] Chemprop multi-task (T4 GPU)
Chemprop pass 1/2: training on 3728 time-window-train mols, predicting on
799 held-out val
== Chemprop cluster=logd_alone seed=0 (primary for ['LogD']) ==
CheMeleon: ✓ loaded from /home/.../.chemprop/chemeleon_mp.pt
(hidden_size=2048)
Total params: 9.3 M (vs 319 K in v3)
```

12.7 v3.1 实测结果 — 出乎意料的失败

T4 上跑了完整 ~6 小时, 官方 eval 实测:

Endpoint	v3 (0.677)	v3.1 实测	Δ vs v3	备注
LogD	0.460	0.532	+0.072 □	CheMeleon + LogD 独训 → 过拟合
KSOL	0.671	0.697	+0.026 □	略差
MLM CLint	0.867	0.863	-0.004	持平
HLM CLint	0.788	0.749	-0.039 □	metab_plus_mppb 7-task 起作用
Caco-2 Efflux	0.817	0.774	-0.043 □	perm_only 4-task 起作用
Caco-2 Papp	0.793	0.772	-0.021 □	同上
MPPB	0.748	0.699	-0.049 □	metab_plus_mppb 7-task 起作用
MBPB	0.457	0.473	+0.016	略差
MGMB	0.488	0.555	+0.067 □	CheMeleon × 222 样本 → 过拟合
Macro	0.677	0.679	+0.002	几乎没变

R^2 : v3 = +0.447, v3.1 = +0.447 (一样)

模式分析:

- 中段 endpoint (HLM / Caco-2 Efflux+Papp / MPPB) 共 4 个改善 0.02-0.05 → HybridADMET 精细分组的功劳
- 极端 endpoint (LogD N=5039 独训 / MGMB N=222 极少) 各退化 0.07 → CheMeleon 的功劳 (反向)

两个改进互相抵消, net 改善 = 0。

12.8 为什么 CheMeleon + 独训 LogD 反而变差?

回看 chemprop 启动日志: CheMeleon 给 chemprop 撑到 9.3 M 参数 (之前 319 K, 30 倍大)。在两种场景下这个大模型会过拟合:

1. LogD 独训 (HybridADMET 配置): 只 1 个 prediction head + 5039 个样本 + 9.3 M 参数 = 严重过拟合。多 task 协训本来是天然的 regularizer, 独训等于撤掉了 regularizer。
2. MGMB 用 all_nine 协训: 虽然多 task 没问题, 但 MGMB 自己只有 222 个样本, 9.3 M 参数在那条 prediction head 上信号比噪声还少。

反思: 当初设计 v3.1 时假定 "foundation model 总是好的", 没考虑到 foundation model 的容量必须跟 task scale 匹配。CheMeleon 2048-dim 大概是在百万分子上预训练的, 下游任务也应该是百万级才匹配。对几千个分子 fine-tune 一个 9 M 模型, 等于让 BERT 在 1000 句话上微调——很容易把预训练表征搞坏。

12.9 当前真实最佳: v3 with 10 seeds + 80 epochs

回看历史:

配置	MA-RAE	R^2	备注
v3 (3-cluster, 5 seeds, 50ep, threshold=1.10)	0.6751	+0.45	
v3 (3-cluster, 10 seeds, 80ep, threshold=1.10)	0.6663	+0.45	历史最佳
v3 (3-cluster, 10 seeds, 80ep, threshold=999)	0.6767	+0.45	
v3.1 (HybridADMET + CheMeleon, threshold=999)	0.6795	+0.45	本节

讽刺的是: 上一次 (认为 CheMeleon 加载成功的) "10 seeds + 80 epochs" 那次 (chemprop 其实是随机初始化), 反而是当前最佳 0.6663。这意味着:

- 更大的 chemprop ensemble (10 seeds vs 5) 确实有用 (-0.005)
- 更多 epochs (80 vs 50) 也有边际帮助
- NNLS threshold=1.10 略好于 999 (在 5-cluster + CheMeleon 之外是这样)
- CheMeleon 没起作用 (无论有没有, 分数差不多)

12.10 v3.1 的 4 条 Lessons Learned

接 §11.6 + §12.9:

1. Foundation model 不一定适合下游任务: CheMeleon 2048-dim 在百万分子上训练得很好, 但下游只有几千个分子时, 过大的容量反而把预训练表征搞坏了。模型大小要匹配数据规模。
Pretrained != automatically good.
2. 独训 vs 协训: HybridADMET 报告"LogD 独训最好", 但他们用的是 Uni-Mol2 + PAMNet 这套机制。换到 chemprop GNN + CheMeleon 初始化的语境, 独训反而 worst case (无 regularization)。Paper 里的设定差异比想象中重要。
3. 看似两个独立改进, 组合后可能互相抵消: HybridADMET grouping 单独可能 +5 个 endpoint 改善, CheMeleon 单独可能 +小 N endpoint 改善, 但组合在 LogD 独训这个具体情境下两者冲突。要先做 ablation 再做 combined experiment。
4. 诚实回看历史: 之前 0.6663 那次"以为 CheMeleon 用上了"其实没用上, 意外暴露了"更大 ensemble + 更多 epochs"的真实价值。实验现象优先于解释——如果一个 trick 看似有效但实际机制错了, 至少现象是真的, 要重新解释而不是丢掉。

v3.1 部分更新时间: 2026-05-07 (最终结果 0.679, 未突破 v3 0.666 历史最佳)

12.8 距离 HybridADMET (0.6) 还差什么

如果 v3.1 落到 ~0.63 而 HybridADMET 是 ~0.6, 剩下的 0.03 差距可能来自:

1. Uni-Mol2 transformer: 他们用了, 我们没用。Uni-Mol2 是 transformer 架构 (vs 我们的 GNN), 在 3D 结构信息上更强。可以加进 v3.2。
2. PAMNet (Physics-Aware Message-passing Network): 他们用了, 我们用的是普通 BondMessagePassing。
3. MACCS / ErG / PubChem fingerprints: 他们提到 fingerprint 分支对 Efflux/Papp 提升明显。我们只用 RDKit + Morgan + Avalon。
4. End-to-end 微调: 他们把 Uni-Mol2 + PAMNet + fingerprint branch 全部一起端到端微调; 我们是 ensemble 各自的 prediction。理论上 stacking ensemble 比联合微调略弱。

12.9 v3.1 新增 Lessons Learned

接 §11.6:

1. 预训练模型的隐藏维度可能远大于你以为的: CheMeleon 是 2048-dim, 比我们随手设的 300-dim 大 7 倍。预训练 checkpoint 的 hyper_parameters 字段必须用, 不能用自己的 d_h 强行覆盖。
2. Foundation 模型在 chemprop v2 里藏在 CLI 源码 (`chemprop/cli/train.py`) 而不是 Python API。grep 源码是定位"功能存在但 API 没暴露"问题的最快方法。
3. 任务分组宁可细不宜粗: v3 的 3 cluster 把 LogD 当通用辅助 task 加到每个 cluster 里, 看似合理 (LogD 跟很多东西相关), 但实际上对 LogD 自己的预测是 negative transfer——HybridADMET 把 LogD 独训反而最好。任务分组应该针对每个 target 单独设计, 不是按"相关性"一刀切。

4. 关掉 NNLS 硬阈值反而更稳：v3 加的 `MAX_LOSS_RATIO=1.10` 在 chemprop val 看着好的 endpoint 上把 LightGBM 等都过滤掉了，等 val→test 偏移一来直接翻车。NNLS 本来就有 0-1 sparsification，足够 robust，不需要额外硬过滤。
 5. GPU 时间不是恒定的：CheMeleon 让 chemprop 大 30 倍，T4 上 per-batch 速度从 42 it/s 掉到 6 it/s。预先做 throughput 测试避免"预计 1 小时实际 6 小时"的尴尬。
-

v3.1 部分更新时间：2026-05-07（运行中，最终数字 TBD）